

AI Assignment

Artificial Intelligence - Questions and Answers

1. Define artificial intelligence? List any two applications of AI.

Artificial Intelligence (AI) is a branch of computer science that enables machines or computer systems to perform tasks that normally require human intelligence, such as learning, reasoning, problem-solving, and decision-making.

Any two applications of AI are:

1. Healthcare - AI is used for disease diagnosis, medical image analysis, and treatment recommendations.
2. Self-driving vehicles - AI helps vehicles detect objects, understand road conditions, and make driving decisions.

2. What is state space search? What is Turing test.

State Space Search: State space search is an AI problem-solving technique in which all possible states of a problem are represented as a state space, and the solution is found by searching from the initial state to the goal state using valid operations.

Turing Test: The Turing Test was proposed by Alan Turing to determine whether a machine can exhibit intelligent behavior similar to a human. If a human evaluator cannot reliably distinguish between the responses of a machine and a human through conversation, the machine is considered to have passed the test.

3. What is a rational agent? What are the different types of rational agents?

A rational agent is an AI system that perceives its environment through sensors and takes actions through actuators to achieve the best possible outcome based on the information available to it.

The main types of rational agents are:

1. Simple Reflex Agent - Acts only on the current percept using predefined condition-action rules.
2. Model-Based Reflex Agent - Maintains an internal model of the environment and uses past information along with the current percept.

3. Goal-Based Agent - Chooses actions based on whether they help achieve a specific goal.
4. Utility-Based Agent - Selects actions that maximize a utility value, helping it choose the most desirable outcome.
5. Learning Agent - Improves its performance over time by learning from experience and feedback.

4. What is meant by heuristic search? Define a heuristic function for 8-tiles problem.

Heuristic Search: Heuristic search is a search technique in Artificial Intelligence that uses additional knowledge or an estimate to guide the search toward the goal more efficiently. It reduces the number of states that need to be explored.

A heuristic function, denoted by $h(n)$, estimates the cost or distance from the current state n to the goal state.

For the 8-tile puzzle, common heuristic functions are:

1. Misplaced Tiles Heuristic: $h(n)$ = number of tiles that are not in their correct positions.
2. Manhattan Distance Heuristic: $h(n)$ = sum of the horizontal and vertical distances of each tile from its goal position.

The blank tile is usually not included in the heuristic calculation.

5. Compare Greedy Best-First Search, A* Search, and IDA* Search algorithms in terms of their working principle, optimality, completeness, time and space complexity, and real-time applications

Feature	Greedy Best-First Search	A* Search	IDA* Search
Working principle	Expands the node that appears closest to the goal using only $h(n)$.	Expands the node with minimum $f(n) = g(n) + h(n)$.	Uses A*'s $f(n) = g(n) + h(n)$ with iterative deepening and increasing thresholds.
Optimality	Generally not optimal.	Optimal when the heuristic satisfies appropriate conditions.	Optimal with an admissible heuristic under standard assumptions.
Completeness	Not always complete in infinite spaces; can be complete in finite graphs with duplicate handling.	Complete when step costs are positive and standard conditions hold.	Complete under standard finite-branching and positive-cost assumptions.
Time complexity	Worst case approximately $O(b^d)$.	Worst case approximately $O(b^d)$.	Exponential, commonly expressed as $O(b^d)$, with repeated-search overhead.

Feature	Greedy Best-First Search	A* Search	IDA* Search
Space complexity	Approximately $O(b^d)$.	Approximately $O(b^d)$.	Approximately $O(d)$.
Main advantage	Often reaches a goal quickly when the heuristic is good.	Balances path cost and heuristic guidance while supporting optimal solutions.	Uses much less memory than A*.
Main disadvantage	Can choose a poor path because it ignores cost already travelled.	High memory consumption.	May re-explore nodes across iterations.
Applications	Route finding, game AI, approximate navigation, recommendation/search problems.	GPS/path planning, robotics, games, network routing, navigation systems.	Large puzzles such as the 15-puzzle, game-state search, robotics, memory-constrained search.

Here, $g(n)$ is the actual cost from the initial state to node n , $h(n)$ is the estimated cost from n to the goal, b is the branching factor, and d is the depth of the solution.

In short: Greedy Best-First Search focuses only on how close the goal appears, A* considers both the cost already travelled and the estimated remaining cost, while IDA* follows the A* evaluation function but uses iterative depth-first search to greatly reduce memory usage.

6. Write A* algorithm and explain with an example?

A* Search Algorithm

A* is an informed search algorithm used to find the least-cost path from a start node to a goal node.

It uses the evaluation function: $f(n) = g(n) + h(n)$

where $g(n)$ is the actual cost from the start node to node n , $h(n)$ is the estimated cost from node n to the goal, and $f(n)$ is the estimated total cost of the path through n .

Algorithm:

1. Put the start node in the OPEN list.
2. Set $g(\text{start}) = 0$ and calculate $f(\text{start}) = g(\text{start}) + h(\text{start})$.
3. Repeat until OPEN is empty:
 - Select the node with the smallest $f(n)$.
 - If it is the goal node, return the path.
 - Otherwise, move it to the CLOSED list.
 - Generate its successor nodes.

- For each successor, calculate its new $g(n)$, calculate $f(n) = g(n) + h(n)$, and add or update the node in OPEN if the new path is better.

4. If OPEN becomes empty before reaching the goal, no solution exists.

Example:

```
S --2--> A --3--> C --2--> G
| ^
|--4--> B --1----/
```

Node	$h(n)$
S	5
A	4
B	3
C	2
G	0

From S to A: $g(A) = 2$, so $f(A) = 2 + 4 = 6$.

From S to B: $g(B) = 4$, so $f(B) = 4 + 3 = 7$.

A* chooses A because it has the smaller f-value.

From A to C: $g(C) = 2 + 3 = 5$, so $f(C) = 5 + 2 = 7$.

If B is expanded, the path to C also has cost $4 + 1 = 5$.

From C to G: $g(G) = 5 + 2 = 7$, so $f(G) = 7 + 0 = 7$.

Therefore, A* reaches the goal with total path cost 7. One optimal path is: S -> A -> C -> G.

Key Point: A* selects the node having the smallest $f(n) = g(n) + h(n)$. Therefore, it considers both the cost already travelled and the estimated remaining cost, which helps it efficiently find an optimal path when an appropriate heuristic is used.